

Lecture 3

Linear Models, Optimisation & Regularisation

AI for Geometry — MM845

Edward Hirst

Instituto de Matemática, Estatística e Computação Científica
Universidade Estadual de Campinas

`ehirst@unicamp.br`

Agosto 2026

Overview

1. Linear Models
 - Regression
 - Classification
2. Optimisation
3. Regularisation & Features
4. Standard Code Workflow
5. Summary

Recap (Lecture 2): Learning = approximation + optimisation + statistics
...minimise a risk functional over a hypothesis space, knowing the measure only through samples.

Guiding question: *In the one setting where everything can be proved, linear models with convex losses, what does the complete theory look like?*

Linear Regression: Orthogonal Projection

Setup

Data $\{(x_i, y_i)\}_{i=1}^N \subset \mathbb{R}^n \times \mathbb{R}$; hypothesis space $\mathcal{H} = \{f_\theta(x) = \theta^\top x\}$

...can introduce constant bias by appending 1 as a dimension of x , then bias b is the last θ_{n+1} .

Least squares:

$$\min_{\theta \in \mathbb{R}^n} L(\theta) = \frac{1}{N} \|X\theta - y\|^2, \quad X \in \mathbb{R}^{N \times n} \text{ (design matrix).}$$

- **Geometry:** $X\theta$ ranges over the column space $\text{col}(X) \subseteq \mathbb{R}^N$; the minimiser is the *orthogonal projection* of y onto $\text{col}(X)$.
- **Normal equations:** $\nabla_\theta L = 0 \iff X^\top X\theta = X^\top y \iff \hat{\theta} = (X^\top X)^{-1} X^\top y$ (when X has full column rank).
- Residual $y - X\hat{\theta} \perp \text{col}(X)$: the error is invisible to the model, by construction.
- In practice: solve by QR/SVD, never by explicit inversion.

Why start here?

- Everything is exact: unique minimiser, closed form, full error analysis.
- Every later model trades away pieces of this certainty for expressive power.

Linear Classification: Logistic Regression

From Regression to Classification

For labels $y \in \{0, 1\}$, model the *probability* via the sigmoid $\sigma(t) = (1 + e^{-t})^{-1} \in [0, 1]$:

$\mathbb{P}(y = 1 \mid x) = \sigma(\theta^\top x)$, ...decision boundary $\{x : \theta^\top x = 0\} \implies$ a hyperplane at $\sigma = 0.5$.

- Maximum likelihood over the data $\implies$ minimise **cross-entropy**:

$$L(\theta) = -\frac{1}{N} \sum_i \left[y_i \log \sigma(\theta^\top x_i) + (1 - y_i) \log(1 - \sigma(\theta^\top x_i)) \right].$$

- No closed form, but L is *convex* $\implies$ gradient methods find the global minimum.
- Outputs are calibrated probabilities, not just labels — confidence clear and interpretable.

Multiclass Classification: Softmax

For K classes, assign one score (logit) to each class: $z_k(x) = \theta_k^\top x$

...and convert these scores into probabilities using: $\mathbb{P}(y = k \mid x) = e^{z_k(x)} / \sum_{j=1}^K e^{z_j(x)}$.

The predicted class is: $\hat{y}(x) = \arg \max_{k \in \{1, \dots, K\}} \mathbb{P}(y = k \mid x)$.

Since the exponential and common denominator preserve the ordering of the logits. Training minimises multiclass cross-entropy:

$$L(\theta) = -\frac{1}{N} \sum_{i=1}^N \log \mathbb{P}(y_i \mid x_i),$$

...so the model increases the probability assigned to the correct class.

Gradient Descent: Discretised Gradient Flow

The iteration

The parameter update process during learning follows:

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t),$$

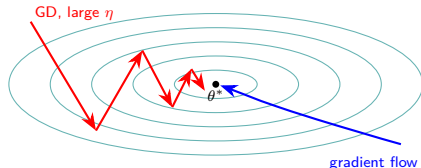
...explicit Euler discretisation for the flow:

$$\dot{\theta} = -\nabla L(\theta),$$

...which dissipates L monotonically:

$$\frac{d}{dt} L = -\|\nabla L\|^2 \leq 0.$$

- η too small:
Slow (many steps per unit flow-time).
- η too large ($> 2/\beta$):
Oscillation/divergence across narrow valleys.



The discrete iterate zig-zags, where the continuous flow glides.

- Ill-conditioning ($\kappa \gg 1$):
Zig-zag across the valley, crawling along it.
- Fixes: momentum, adaptive steps, preconditioning $\mapsto$ Adam (Lecture 4).

Convexity: When Optimisation Is Easy

Definitions

- L is **convex** if, for $\lambda \in [0, 1]$, $L(\lambda\theta + (1-\lambda)\theta') \leq \lambda L(\theta) + (1-\lambda)L(\theta')$.
- L is **β -smooth** if ∇L is β -Lipschitz, $\|\nabla L(\theta) - \nabla L(\theta')\| \leq \beta\|\theta - \theta'\|$.
- L is **α -strongly convex** if $L(\theta) - \frac{\alpha}{2}\|\theta\|^2$ is convex.

- Convex: every local minimum is global; first-order condition $\nabla L(\theta^*) = 0$ is sufficient.
- Smooth + strongly convex: then $\alpha I \preceq \nabla^2 L(\theta) \preceq \beta I$, such that gradient descent with $\eta = 1/\beta$ converges *linearly*. Using the **condition number** $\kappa = \beta/\alpha$, then

$$L(\theta_t) - L^* \leq \left(1 - \frac{1}{\kappa}\right)^t (L(\theta_0) - L^*),$$

... $\kappa \approx 1 \implies$ quick convergence as all update directions good.

- For least squares, $\kappa = \frac{\lambda_{max}(X^\top X)}{\lambda_{min}(X^\top X)}$: the eccentricity of the elliptical level sets.
...normalising features (Lecture 2) is literally rounding the ellipses.

Why care, if deep learning is nonconvex?

- Near minima, the quadratic (convex) picture is locally accurate.
- Convex analysis is the canonical baseline that non-convex analysis can perturb from.

Stochastic Gradient Descent (SGD)

The estimator

For $L(\theta) = \frac{1}{N} \sum_i \ell_i(\theta)$ with N huge, sample a *minibatch* $B \subset \{1, \dots, N\}$:

$$\mathbb{E}[\text{minibatch gradient}] = \nabla L(\theta_t), \quad \implies \quad \theta_{t+1} = \theta_t - \eta \frac{1}{|B|} \sum_{i \in B} \nabla \ell_i(\theta_t).$$

- Cost per step: $O(|B|)$ instead of $O(N)$ — the only reason training on millions of samples is feasible, as can get the batch of data into GPU memory.
- The gradient noise is not merely tolerated: it helps escape saddle points and biases towards *flat* minima, which empirically generalise better.
- Convergence requires decaying steps ($\sum \eta_t = \infty$, $\sum \eta_t^2 < \infty$) or averaging; in practice piecewise/cosine decay schedules $\mapsto$ the schedules are the ‘art’ of AI.
- Batch size trades gradient variance against parallel hardware efficiency; typical $|B| \in [16, 1024]$ (powers of 2 optimal for matmul batching on hardware).

Implicit regularisation

SGD does not explore the loss landscape uniformly:

...minibatch noise is larger in directions where individual sample gradients disagree. This anisotropic noise pushes away from sharp minima and towards broader regions of low loss.

$\mapsto$ So the optimisation algorithm itself influences which solution is selected.

Regularisation: Ridge and Lasso

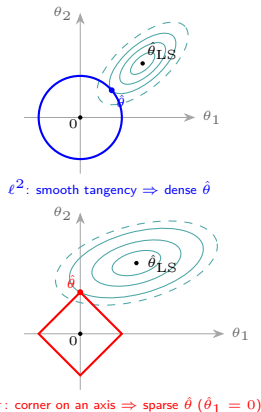
Penalised least squares

To regularise learning, we add a penalty term to the loss:

$$L(\theta) = \|X\theta - y\|^2 + \lambda\Omega(\theta), \quad \Omega = \begin{cases} \|\theta\|_2^2 & \text{ridge} \\ \|\theta\|_1 & \text{lasso} \end{cases}$$

- **Ridge** (ℓ^2 regularisation): constrains θ to a smooth, round ℓ^2 ball. The elliptical level sets of the data-fitting loss typically touch this ball at a smooth point, so coefficients are continuously shrunk towards zero but are rarely set exactly to zero; less impactful directions are shrunk most strongly.
- **Lasso** (ℓ^1 regularisation): constrains θ to a diamond-shaped ℓ^1 ball with corners on the coordinate axes. Loss level sets often first intersect the constraint at one of these corners, forcing some coordinates to vanish exactly and therefore performing automatic feature selection.

Always tune λ on the validation set (never on test data).



Level sets (ellipses) of $\|X\theta - y\|^2$ about unpenalised minimiser $\hat{\theta}_{LS}$.
The regularised $\hat{\theta}$ sits where the lowest level set reaches the constraint ball.
Dashed sets already overlap it.

Feature Maps: Linear Models, Nonlinear Power

The trick

Precompose with a nonlinear **feature map** $\phi : X \rightarrow \mathbb{R}^D$ and fit a linear model in feature space:

$$f_{\theta}(x) = \theta^{\top} \phi(x).$$

Still linear in θ — so with a convex loss the problem stays convex; all the theory survives.

- **Polynomial features:** $\phi(x) = (1, x_1, x_2, x_1^2, x_1x_2, \dots)$, fits algebraic hypersurfaces; conics separate what lines cannot.
- **Geometric features:** symmetry-invariant quantities you compute — volumes, intersection numbers, spectra, Betti numbers. Encoding domain knowledge in ϕ is often worth more than any architecture choice.
- Scaling problem: polynomial features blow up combinatorially, so embedding expensive.
→ **kernel trick** (Lecture 8) evaluates $\langle \phi(x), \phi(x') \rangle$ without constructing ϕ .
- Deep learning (Lecture 4): *learn* ϕ instead of designing it.

Design principle

Baseline every problem with a linear model on thoughtful features before reaching for deep learning — it is fast, interpretable, and surprisingly often sufficient!

PyTorch in One Slide

AI training = Tensors + autograd + optimiser

PyTorch tensors are NumPy arrays that (i) live on CPU/GPU and (ii) record operations so gradients come free via 'autograd' (Lecture 4).

Example (The canonical training loop)

```
import torch
model = torch.nn.Linear(n_features, 1)          #  $f(x) = wx + b$ 
opt = torch.optim.SGD(model.parameters(), lr=1e-2)
lossf = torch.nn.MSELoss()

for epoch in range(200):
    opt.zero_grad()                             # reset stored gradients
    loss = lossf(model(X), y)                   # forward pass
    loss.backward()                             # autodiff: compute grad L
    opt.step()                                  #  $\theta \leftarrow \theta - \eta * \text{grad}$ 
```

- Every model in this course — up to billion-parameter transformers — trains with exactly this loop; only model, lossf, and opt change.

Simplicity is interpretability

- When the model is transparent, its learned weights are readable mathematics.
- Large coefficients point at the parameters / invariants that matter.

Summary

Key points

- Canonical Regression $\implies$ Least squares (orthogonal projection).
Canonical Classification $\implies$ Logistic regression (hyperplane classification).
- Convexity + smoothness give global quantitative convergence for gradient descent; the condition number, κ , is the geometry of the level sets for loss minimisation.
- SGD makes large-scale learning feasible; its noise is a feature (escape saddles, prefers flat minima) that needs to be tuned.
- Ridge shrinks, lasso sparsifies — the ℓ^1/ℓ^2 ball picture; regularisation strength, λ , is tuned on validation data.
- Feature maps carry linear theory into non-linear territory. Choosing features appropriate to the problem is hugely impactful $\mapsto$ *geometric intuition*.

Next

Tutorial 3: Linear and logistic regression on geometric datasets in PyTorch.

Lecture 4: Neural networks — large non-linear ansätze (learning feature maps directly).